Trabalho Prático

Alunos: João Victor Póvoa França e Otávio Nardini

Atividade 3 - Implementar Encriptador e Decriptador.

1. Estratégia e Técnica Utilizada

A arquitetura do projeto foi estruturada como um ecossistema de microsserviços e bibliotecas independentes, operando em um monorepo para isolar responsabilidades de criptografia, telemetria e interface. O tráfego de dados entre as Máquinas Virtuais (VMs) e a execução dos algoritmos exigem um entendimento profundo da manipulação de *bytes* em memória e da comunicação entre processos.

Link da aplicação:https://github.com/Joaolto/encrypt-decrypt-sign/tree/main

A comunicação *End-to-End* entre as VMs virtuais começa na interface construída em Next.js. Quando um teste é disparado, o *front-end* utiliza o módulo nativo child_process do Node.js para verificar via terminal se a porta da VM alvo está acessível. Confirmada a conexão, o *payload* (a mensagem em texto claro) é injetado em uma requisição HTTP para a rota /encrypt do *back-end* (Fastify). Esse serviço importa funções puras do pacote isolado crypto-core. Após a transformação criptográfica, o resultado é persistido na memória RAM através do módulo logger.

A cifra ofuscada é devolvida e renderiza a transação visualmente. Em seguida, a aplicação envia o lixo criptográfico para a rota /decrypt, com a recepção pelo destinatário, onde o processo é revertido para exibir o texto original. Simultaneamente, requisições de telemetria ocorrem a cada 1000ms na rota /logs para atualizar o console visual e ter os logs visualmente inspecionados.

2. Funcionamento do Algoritmo: Processo de Encriptação

A base da segurança do sistema reside na aplicação combinada de dois conceitos clássicos de criptografia: ***substituição e transposição***.

A aplicação recebe o texto digitado pelo usuário, faz a leitura dessa entrada em texto claro e realiza as transformações necessárias para exibir o texto cifrado ao final do processo.

A **substituição** é uma técnica algébrica onde cada caractere do texto claro é substituído por outro valor, alterando a identidade da informação original. Computacionalmente, isso é feito extraindo o valor inteiro do caractere na tabela ASCII (utilizando métodos como charCodeAt()) e aplicando uma função matemática sobre esse *byte*. A fórmula linear atômica é definida por ***C(x) = x + Shift***. Após a soma ou subtração escalar, o motor V8 do JavaScript uti liza String.fromCharCode() para converter o novo número de volta em uma *string* codificada.

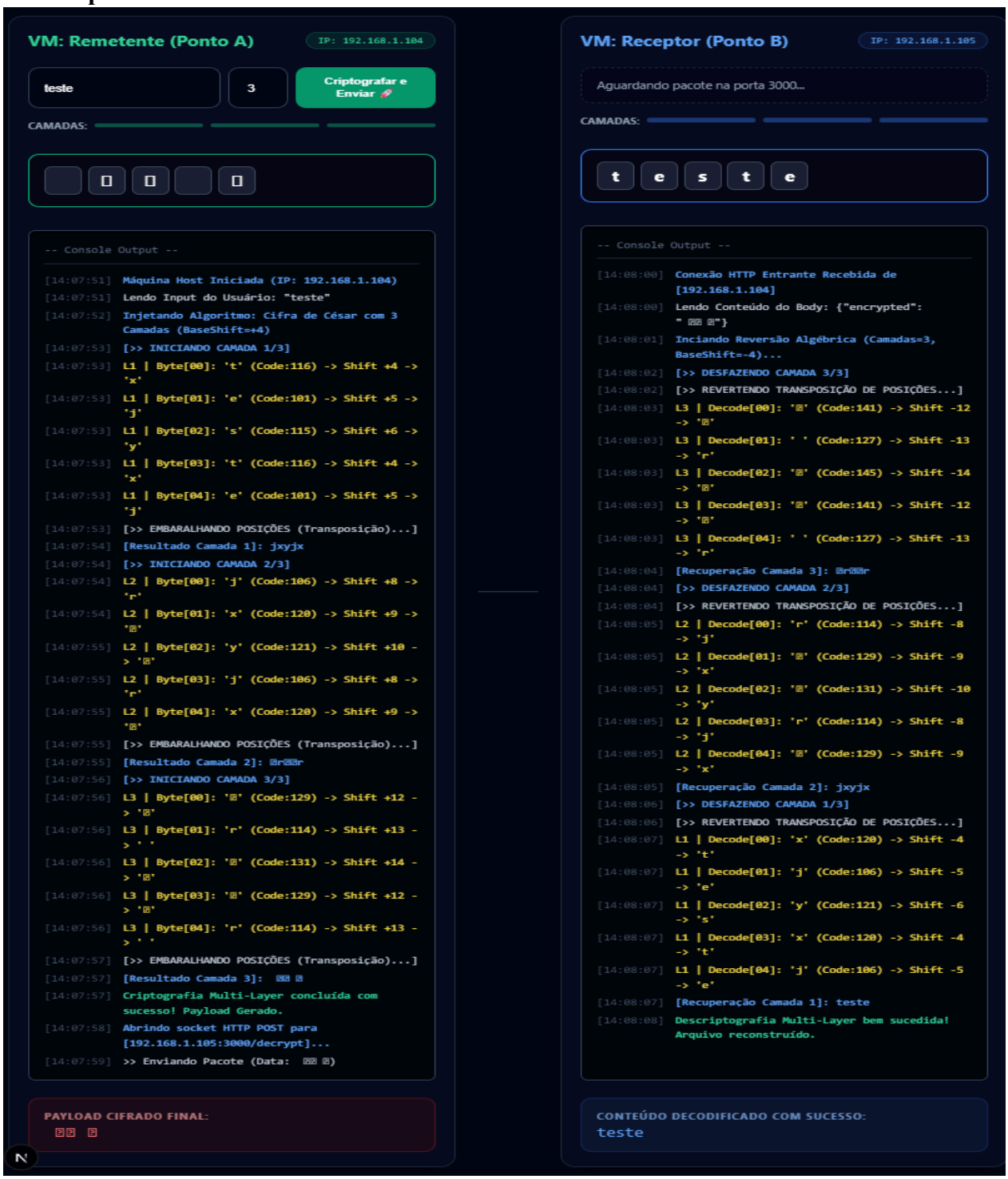
A **transposição**, por outro lado, atua na geometria da estrutura de dados. Em vez de alterar o valor binário do caractere, o algoritmo reorganiza a sua posição espacial dentro da matriz ou *string*. No contexto deste projeto, a transposição ocorre de maneira brutal através da inversão integral (*reverse*) do arranjo de caracteres. Essa quebra posicional é fundamental para mitigar ataques baseados em análise de frequência sintática, pois destrói a coesão léxica das palavras originais

```
// ==========================================
// 1. CIFRA (Caesar Cipher)
// ==========================================

export function encryptCaesar(message, shifts = 4) {
  return message.split("").map((caractere) => {
    const codigoCaractere = caractere.charCodeAt(0);
    return String.fromCharCode(codigoCaractere + shifts);
  }).join("");
}

export function decryptCaesar(encryptedMessage, shifts = 4) {
  return encryptedMessage.split("").map((caractere) => {
    const codigoCaractere = caractere.charCodeAt(0);
    return String.fromCharCode(codigoCaractere - shifts);
  }).join("");
}
```

3. O Conceito de Camadas (Multi-Layer Cascading) e o Código

Para elevar o grau de complexidade matemática e o esquema de chaves utilizado, o sistema não se apoia em uma cifra linear única. Em vez disso, implementa uma arquitetura de Múltiplas Camadas Iterativas ***(Multi-Layer Cascading)***.

As camadas funcionam como ciclos sucessivos de *loops* de processamento. A saída cifrada da primeira camada torna-se o Fator de Entrada para a segunda, e assim sucessivamente. Em cada iteração de ***camada (L)***, o algoritmo aplica um deslocamento dinâmico que não é mais um valor estático, mas sim uma matriz de injeção de distorção baseada no índice posicional (i) do caractere: ***D(L,i) = Base x L) + (i (mod 3))***. Logo após a substituição dinâmica de todos os *bytes* daquela camada, o código dispara a função de transposição em toda a *string*, embaralhando o vetor antes que a próxima camada inicie o processamento.

```javascript
// ==========================================
// 1.5 CIFRA EM CASCATA (Complexidade Multi-Layer)
// ==========================================
export function encryptCaesarCascade(message, baseShift = 4, layers = 3) {
  let result = message;

  for (let layer = 1; layer <= layers; layer++) {
    // 1. Cifragem com Fator Dinâmico: BaseShift * CamadaAtual
    const dynamicShift = baseShift * layer;
    let temp = "";

    for (let i = 0; i < result.length; i++) {
      // Complexidade Algorítmica: O shift depende do índice para evitar matemática linear (A+A = 2A)
      const charCode = result.charCodeAt(i);
      const positionalShift = dynamicShift + (i % 3); // Opcional: injetando distorção posicional
      temp += String.fromCharCode(charCode + positionalShift);
    }

    // 2. Transposição (Reverse string) antes de descer pra próxima camada
    result = temp.split("").reverse().join("");
  }

  return result;
}

export function decryptCaesarCascade(encryptedMessage, baseShift = 4, layers = 3) {
  let result = encryptedMessage;

  // A Reversão DEVE Ocorrer na ordem inversa (Da última camada até a primeira)
  for (let layer = layers; layer >= 1; layer--) {
    // 1. Desfazer Transposição (Invertendo a String novamente)
    result = result.split("").reverse().join("");

    // 2. Desfazer Fator Dinâmico: BaseShift * CamadaAtual
    const dynamicShift = baseShift * layer;
    let temp = "";

    for (let i = 0; i < result.length; i++) {
      const charCode = result.charCodeAt(i);
      const positionalShift = dynamicShift + (i % 3);
      temp += String.fromCharCode(charCode - positionalShift);
    }

    result = temp;
  }

  return result;
}
```

4. Exemplo de teste: